

# CSCI 4170 - Project in AI and ML

## Homework 3 - Summer 2025

Haruto Suzuki

Due date: 11:59 pm, June. 19th

### Part 1

In this part you will implement a neural network from scratch. You cannot use any existing Deep Learning Framework. You can utilize NumPy and Pandas libraries to perform efficient calculations.

Write a class called `NeuralNetwork` that has at least the following methods (you are free to add your own methods too):

- a. Initialization method.
- b. Forward propagation method that performs forward propagation calculations.
- c. Backward propagation method that implements the backpropagation algorithm discussed in class.
- d. Train method that includes the code for gradient descent.
- e. Cost method that calculates the loss function.
- f. Predict method that calculates the predictions for the test set.

Test your `NeuralNetwork` Class with the dataset you selected. If the dataset is big, you may notice inefficiencies in runtime. Try incorporating different versions of gradient descent to improve that (Minibatch, Stochastic etc.). You may choose to use only a subset of your data for this task (or any other technique). Explain which technique you followed and why.

### A.

The original dataset contains 15,000 samples for each of the five classes, totaling 75,000 instances. To address the concern that large datasets can lead to inefficiencies in runtime, I initially performed random sampling to reduce the dataset size. Specifically, I selected 2,000 samples from each of the five classes, resulting in a training set of 10,000 samples and an additional 10,000 for validation. This sampling allowed for a manageable yet representative subset of the data.

I first implemented *batch gradient descent*, which processes the entire training set to compute gradients before updating the weights. This method completed 100 epochs in approximately

80 seconds and achieved a validation accuracy of 0.9133. While relatively efficient, batch gradient descent tends to average out finer details in the data due to its use of the entire dataset per update.

To improve both performance and accuracy, I experimented with *mini-batch gradient descent*. Although each epoch took slightly longer, due to more frequent gradient calculations and weight updates, the algorithm converged faster in fewer epochs. After just 30 epochs (taking about 40 seconds), it reached a higher validation accuracy of 0.9708. The increased accuracy can be attributed to the fact that mini-batch gradient descent performs multiple updates using a portion of data per epoch, allowing the model to capture more granular patterns in the training data. Additionally, mini-batches introduce a degree of noise into the gradient updates, which can help escape shallow local minima and improve generalization.

## Part 2

In this part you will implement a 2-layer neural network using any Deep Learning Framework. You should pick a Deep Learning Framework that you would like to use to implement your 2-layer Neural Network.

### Task 1 (5 points)

Assuming you are not familiar with the framework, in this part of the homework you will present your research describing the resources you used to learn the framework (must include links to all resources). Clearly explain why you needed a particular resource for implementing a 2-layer Neural Network (NN). (Consider how you will keep track of all the computations in a NN, i.e., what libraries/tools do you need within this framework.)

I know PyTorch, but I reviewed the basics again using the following:

1. Build the Neural Network from the official PyTorch documentation.  
This tutorial helped me understand the general structure of defining a neural network in PyTorch using the `nn.Module` class. It provided a clear example of how to declare layers, initialize parameters, and implement the `forward()` method, which is central to forward propagation.
2. PyTorch Forum Discussion: Batch Processing in `nn.Linear`.  
This thread clarified how batches of inputs are handled internally by `nn.Linear` layers. It helped me understand how PyTorch efficiently handles batched matrix multiplications, and how tensor shapes should be aligned when feeding inputs through the network during training.
3. Optimizing Model Parameters from the official PyTorch documentation.  
This tutorial was critical for implementing the training loop, including how to compute the loss, call `.backward()` for backpropagation, and update model parameters using optimizers like SGD or Adam. It demonstrated the complete process of combining forward and backward passes with optimization in practice.

## Task 2 (35 points)

Once you have figured out the resources you need for the project, you should design and implement your project. The project must include the following steps (it's not limited to these steps):

1. Perform Exploratory Data Analysis (can include data cleaning, visualization, etc.).

The dataset consisted of rice grain images categorized into five classes: **Arborio**, **Basmati**, **Ipsala**, **Jasmine**, and **Karacadag**. Images were converted to grayscale and resized to  $50 \times 50$  to reduce dimensionality. This resulted in flattened 2500-dimensional input vectors. Sample counts across classes were verified to be balanced before proceeding.

2. Perform a train-dev-test split.

I randomly selected 2,000 images per class to form a training set of 10,000 samples. Another 10,000 samples were set aside for validation. Random seeding was used to ensure reproducibility. The split was implemented within the custom **Dataset** class.

3. Implement forward propagation. Clearly describe the activation functions and other hyperparameters used.

The network was defined using PyTorch's **nn.Sequential** within a custom **Network** class. The hyperparameters I will be adjusting are numbers of hidden layers, epochs, and batch size. When using hidden layers, each hidden unit used the ReLU activation function. The output layer used logits, as the **CrossEntropyLoss** function internally applies **softmax**. The input size was 2500 and the output size was 5.

4. Compute the final cost function.

The loss function used was **nn.CrossEntropyLoss**, which is appropriate for multiclass classification tasks. It combines **LogSoftmax** and **NLLLoss** in one function.

5. Implement gradient descent (any variant such as Adam, RMSProp, etc.) to train the model. Consider regularization if appropriate.

I used the Adam optimizer (**torch.optim.Adam**) with a learning rate of  $1 \times 10^{-4}$ . Adam was selected for its adaptive learning rate and faster convergence properties, especially helpful for networks with more parameters or sparse gradients.

6. Experiment with normalized inputs. Comment on model performance with and without normalization.

Input image pixel values were scaled to the  $[0, 1]$  range by dividing by 255. I tested both normalized and unnormalized data. Normalization produced better convergence and final accuracy in some cases. The loss and accuracy curves were generally more stable when the data was normalized. Without normalization, the loss often exhibited instability—frequent fluctuations or even increases during training. Similarly, accuracy was more volatile.

With my implementation:

- Using **batch gradient descent** on **non-normalized data**, I was able to train deep networks (2 hidden layers) with good validation accuracy.
- Using **batch gradient descent** on **normalized data**, the model required deeper architectures (5 hidden layers) to match similar performance.
- Using **mini-batch gradient descent**, normalized data yielded better validation accuracy than non-normalized data.
- Additionally, increasing the number of hidden layers with non-normalized data led to gradient explosion or vanishing, causing the model to fail to train. Normalization helped mitigate this issue and allowed deeper models to converge. It is worth to note that non-normalized data was able to achieve high accuracy with less number of hidden layers while the normalized data wasn't able to.

With the PyTorch implementation:

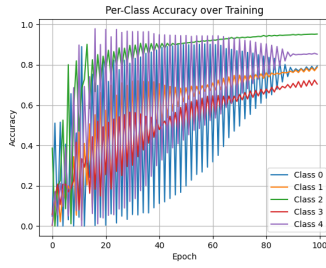
- Both normalized and non-normalized data worked well with 2 hidden layers.
- However, the non-normalized dataset gave slightly better validation accuracy in this case.
- There was no gradient explosion or vanishing observed up to 5 hidden layers and it did gave a higher accuracy.

For the below figures, non-normalized data was trained with 2 hidden layers for all three approaches. For normalized data, my implementation needed 5 hidden layers while I kept the PyTorch implementation the same.

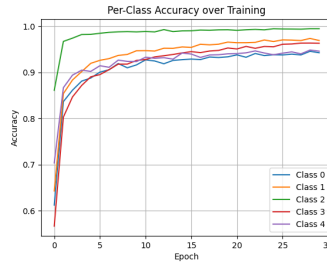
As you can observe, the loss for normalized data is a lot more smoother and constantly decreasing while the non-normalized data is unstable causing some side effects in accuracy as well. In terms of the accuracy, non-normalized data is unstable for my Batch GD and PyTorch, and it can be attributed to unsteady loss. In contrast, normalized data achieved steady increase in the accuracy for all models.

## Comparison of Accuracy and Loss Curves

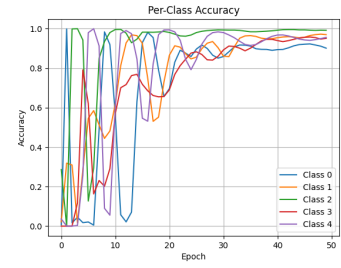
### Non-Normalized Data Results



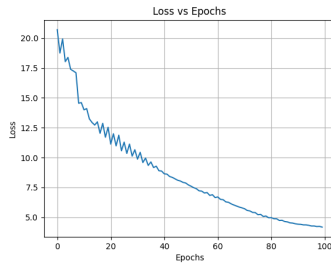
(a) Accuracy-Batch(Own)



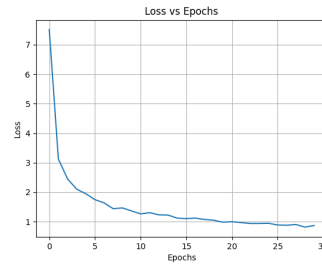
(b) Acc-Mini-Batch(Own)



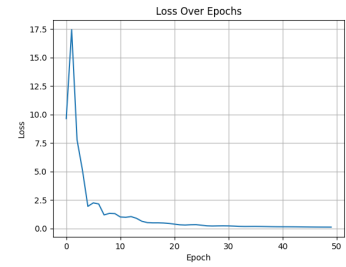
(c) Acc-PyTorch(Batch)



(d) Loss-Batch(Own)



(e) Loss-Mini-Batch(Own)



(f) Loss-PyTorch(Batch)

Figure 1: Training curves on non-normalized data.

## Normalized Data Results

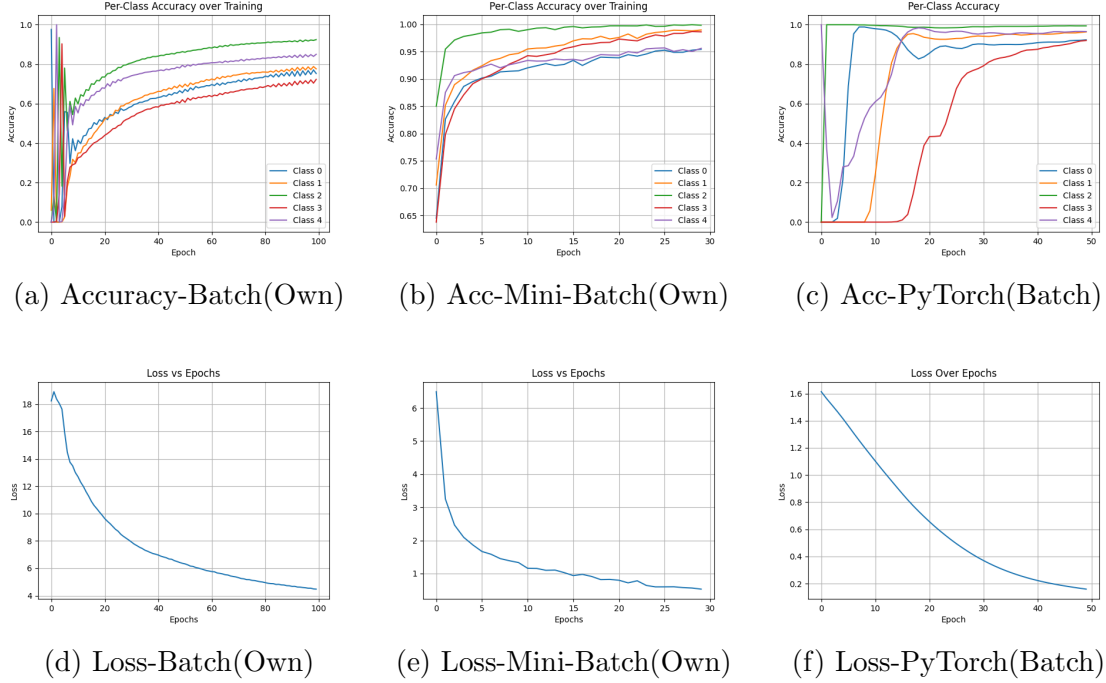


Figure 2: Training curves on normalized data.

7. Present the results using the test set.

### Results on the Test Set

- **Validation Accuracy:** 0.9471
- **Validation Loss:** 0.1654
- **Per-Class Accuracy:**
  - Class 0 (Arborio): 0.9205
  - Class 1 (Basmati): 0.9555
  - Class 2 (Ipsala): 0.9925
  - Class 3 (Jasmine): 0.9100
  - Class 4 (Karacadag): 0.9570

8. Hyperparameter Tuning

I performed hyperparameter tuning using a grid search across hidden layers  $\{1, 3, 5\}$ , learning rates  $\{10^{-3}, 10^{-4}\}$ , and epoch counts  $\{50, 100\}$ . The best validation accuracy (96.88%) was achieved using 5 hidden layers, a learning rate of  $10^{-3}$ , and 100 epochs. In general, models trained with a learning rate of  $10^{-3}$  consistently outperformed those trained with  $10^{-4}$ , especially as the depth of the network increased. Training time scaled proportionally with the number of epochs and layers, with the most complex

configuration taking approximately 41.5 seconds. Configurations using  $10^{-4}$  often suffered from high validation loss and poor accuracy, especially with deeper networks, likely due to vanishing gradients. Below is a summary of the key hyperparameter settings and their resulting validation accuracy.

| Hidden Layers | Learning Rate | Epochs | Validation Accuracy |
|---------------|---------------|--------|---------------------|
| 1             | 1e-3          | 50     | 0.9325              |
| 1             | 1e-3          | 100    | 0.9575              |
| 1             | 1e-4          | 50     | 0.7590              |
| 1             | 1e-4          | 100    | 0.8864              |
| 3             | 1e-3          | 50     | 0.9643              |
| 3             | 1e-3          | 100    | 0.9682              |
| 3             | 1e-4          | 50     | 0.5337              |
| 3             | 1e-4          | 100    | 0.8498              |
| 5             | 1e-3          | 50     | 0.9581              |
| 5             | 1e-3          | 100    | <b>0.9688</b>       |
| 5             | 1e-4          | 50     | 0.4073              |
| 5             | 1e-4          | 100    | 0.7580              |

### Task 3 (10 points)

In task 2 describe how you selected the hyperparameters. What was the rationale behind the technique you used? Did you use regularization? Why, or why not? Did you use an optimization algorithm? Why or why not?

To select the hyperparameters for my model, I performed a grid search over the number of hidden layers  $\{1, 3, 5\}$ , learning rates  $\{1e^{-3}, 1e^{-4}\}$ , and number of training epochs  $\{50, 100\}$ . This approach allowed me to systematically evaluate how different configurations affect validation accuracy and training time. I chose this technique because it is simple, exhaustive over a small search space, and effective for models with a limited number of tunable parameters.

I did not include regularization (such as weight decay or dropout) in this iteration, because my model already showed strong performance on the validation set without overfitting. However, if I noticed signs of overfitting (e.g., training accuracy much higher than validation accuracy), I would consider adding dropout or L2 regularization to mitigate that.

For optimization, I used the Adam optimizer instead of vanilla gradient descent. Adam adapts the learning rate for each parameter and combines the advantages of both AdaGrad and RMSProp. This helped the model converge faster and more reliably, especially in deeper networks where learning rate scheduling and adaptive updates become critical.